Al-Shahariar
IT-24057

```
import java.util.concurrent.atomic.AtomicLong;
Public class Shahariar_Thread implements
Runnable {

//Static conters. one copy shared by all
 object

Static AtomicLong safestatic = new AtomicLong
 (0);
Static long unsafestatic = 0;

//Non Static counters. each object has its
 own copy

long instancecount = 0;
 int k;
 boolean SafeMode;
Shahariar_Thread (int k, boolean safeMode) {
 this.k = k;
 this.safeMode = safeMode;
    }
```

Al-Shahariar
IT-24057

```
Public void run(){
for(int i=0; i<K; i++){
 if (SafeMode) {
 SafeStatic.incrementAndGet();
 }
 else{
 unsafestatic ++ i
 }
 instance count++ i;
 }
 }

public Static void main(String[] args) throws
InterruptedException {
if(args.length != 3){
System.out.println("usage. Java Shahariar-
Thread <N> <K> <true|false>");
 return;
```

```
}
int n = Integer.parseInt(args[0]);
int k = Integer.parseInt(args[1]);
boolean safeMode = Boolean.parseBoolean
(args[2]);
Shahariar_Thread[] task=new Shahariar_
Thread[n];
Thread[] thread = new Thread[n];
for(int i=0; i<n; i++){
tasks[i] = new Shahariar_Thread(k,
safeMode);
thread[i] = new Thread(tasks[i]);
}
for(int i=0; i<n; i++){
threads[i].start();
}
```

CamScanner

Al-Shahariar
IT-24057

```
for (int i=0; i<n; i++){
threads[i].join();
}
long e =(long) n*k;
long S = SafeMode? Safestatic.get():
  unsafestatic;
long SumI=0;
for (int i=0; i<n; i++){
SumI+ = task[i].instancecount;
}
long d = Math.abs (S - SumI);
String P;
if (SumI == 0) {
 P = (S==0)? "0.0000%.": "undefined";
}
else{
```

Al-Shahariar
IT-24057

```
P = String.format("%.4f%%", (d*100.0)/
    SumI);
}
System.out.println("Mode: "+(SafeMode?"
 true (AtomicLong, thread-safe)": "false (long,
 unsynchronized)"));
System.out.println("Thread (N): "+n);
System.out.println("Increments (K): "+K);
System.out.println("Expected (E= N*K): "+e);
System.out.println("Static count (S): "+S);
System.out.println("Non-Static (I): "+SumI);
System.out.println("Abs difference (D): "+d);
System.out.println("Difference (P): "+P);
}
}
```